

Assignment 4—Objects and Subclasses

Due: 6 May

We presented a Java class `Contact` that represents a contact in a contacts database, and we wrote a Java class `Contacts` that uses an array of `Contact` to represent the database.

The `Contacts` class has a lookup function that is not optimally useful. You can only look up a `Contact` if you know the exact first and last name.

We want to make lookup easier by creating more flexible subclasses of `Contact` and `Contacts`.

Problem 1

Create a subclass `ContactEZ` of `Contact` that adds a method *contains* with this interface:

```
/*
    Function: contains
    Returns true if the argument info appears as a
    substring of the name, surname, email, phone or
    address
*/
public boolean contains(String info)
```

Hint: consider using the `String.indexOf` method.

Override the *equals* method so that two `ContactEZ` objects are considered equal only if *all* the information in the contacts is the same: name, surname, email, phone and address (if those fields exist).

Problem 2

Create a subclass `ContactsEZ` of `Contacts`. `ContactsEZ` should overload the *lookup* method to add this variant:

```
/*
    Function: lookup
    Return an array of all contacts that contain
    the info string (contain in the sense of the
    contains() method from the ContactEZ class.)
    Returns null if no contacts contain the info.
*/
public ContactEZ [] lookup(String info)
```

This function may present some challenges to write. You can collect the matching contacts in a temporary array. Once you know how many matches you have, you can create an array of the correct size to return and copy the matches to it from the temporary array.

Write a `Test.java` class with a `main()` function that demonstrates that `ContactEZ` and `ContactsEZ` are working correctly.

Do not modify the original `Contact.java` or `Contacts.java` files.

Make sure your name is in a comment at the top of each java file.

Submit a .zip file containing the three .java files.